Tarea Programada

Modelo Conjunto (Unión, Intersección y Diferencia)

Tercera Etapa

Análisis de Algoritmos y Estructuras de Datos

Docente: Braulio Jose Solano Rojas

Sebastian Arroyo Marin C5l364
Julián F. Vargas Redondo C4K773

Lenguaje utilizado: Python

1. Resumen de hallazgos

Se midieron las siete estructuras del Modelo Conjunto (lista por punteros, lista por arreglos, tabla hash, ABB por punteros, ABB por vector heap, trie por punteros y trie por arreglos) con cuatro tipos de conjuntos y tamaños de N entre 10 y 50 000 elementos por conjunto. Todas las operaciones (Insert, Member, Delete, Print, Union, Intersection y Difference) devolvieron resultados idénticos a los de `set` de Python en todas las mediciones verificadas.

- **ABB por punteros** fue la estructura más rápida y estable con datos en orden aleatorio: Insert y Member cercanos a 1 µs hasta N = 50 000 y Union de 10 000 + 10 000 elementos en unos 16 ms. Su punto débil es la inserción en orden ascendente: el árbol se vuelve una cadena y cada operación pasa de 0,8 µs a 222 µs con N = 10 000.
- **Tabla hash**: con la tabla fija de 10 posiciones del código actual, el tiempo crece en proporción a N/10. Con N = 50 000, Insert cuesta 42 µs y Union 2,6 s. Una variante de referencia con tamaño proporcional a N lo baja a 0,69 µs y 71 ms. El código no implementa la redistribución que describe la documentación.
- **Listas** (punteros y arreglos): Insert, Member y Delete son lineales y Union, Intersection y Difference son cuadráticas porque insertan elemento por elemento. Con N = 10 000, Union tarda entre 3,8 s y 6,9 s. Con N = 50 000 la estimación superó el presupuesto de tiempo y no se midió.
- **Tries**: sus operaciones dependen de la longitud de la cadena y casi no de N (Member entre 1 y 3 µs hasta N = 10 000), pero usan entre 1,2 KB y 7,6 KB por elemento, de 20 a 130 veces más memoria que una lista por arreglos.
- **ABB por vector heap**: es inviable en la práctica. El vector crece de forma exponencial con la altura del árbol. Con datos aleatorios falla en N = 1 000 (más de 4,19 millones de posiciones) y las operaciones de conjunto ya son inviables con N = 20 (Union) o N = 50 (Intersection y Difference).

2. Metodología

Se desarrolló el programa analisis.py, que importa directamente las clases de las dos etapas anteriores sin modificarlas y escribe sus resultados en tiempos.csv y espacios.csv, además de los gráficos.

Estructuras evaluadas. Las siete del proyecto.

Tipos de conjuntos. A y B tienen N elementos cada uno y comparten el 50 % de sus elementos. Se usaron cuatro tipos:

- **Aleatorio 8**: cadenas alfanuméricas aleatorias de 8 caracteres, insertadas en orden aleatorio.
- **Aleatorio 20**: cadenas aleatorias de 20 caracteres (longitud máxima permitida, peor caso para los tries).
- **Prefijo común**: 15 caracteres iguales y 5 distintos (mejor caso de compartición para los tries y comparaciones de cadenas más costosas).
- **Ordenado**: las mismas cadenas del tipo Aleatorio 8, pero insertadas en orden ascendente (peor caso para los árboles de búsqueda).

Tamaños. N = 10, 20, 30, 50, 100, 200, 500, 1 000, 5 000, 10 000 y 50 000. Los tries se limitaron a N = 10 000 para no agotar la memoria del equipo de pruebas.

Tiempo. Se usa `time.perf_counter` con el recolector de basura desactivado durante cada medición. Insert se mide sobre los N elementos y se informa por elemento. Member (presente y ausente) usa una muestra de hasta 1 000 elementos y Delete una de hasta 300; ambos se informan por operación. Print se mide con la salida redirigida a memoria, para no medir la consola. Union, Intersection y Difference se miden completas. Se repite cada medición 5 veces ($N \leq 1\,000$), 3 veces ($N \leq 10\,000$) o 1 vez y se reporta la mediana.

Espacio. Se mide con `tracemalloc` la memoria que ocupa la estructura al insertar los N elementos. Para las estructuras que guardan las cadenas completas se suma el tamaño de cada cadena, de modo que la comparación con los tries (que guardan las cadenas dentro del camino) sea justa. Por ello los valores de ”bytes por elemento” incluyen la cadena.

Control de duración. Antes de medir un tamaño mayor, el programa estima el tiempo suponiendo comportamiento cuadrático a partir de la medición anterior. Si supera 15 s, la operación se omite y se deja registrada. Para el ABB por vector heap, el programa simula la inserción para calcular cuántas posiciones necesitaría el vector y, si supera 4 194 304, marca la medición como inviable en lugar de intentar reservar esa memoria.

Entorno de pruebas. Python 3.12.3, un Ryzen 7 5700G y 16 GB de RAM. Los valores absolutos cambiarán en otro equipo, pero las tendencias y las relaciones entre estructuras deberían mantenerse.

3. Resultados de tiempo (conjunto aleatorio de 8 caracteres)

Las tablas 1 y 2 muestran el costo por elemento en microsegundos. «Inviable» indica que el vector del ABB heap necesitaría más de 4,19 millones de posiciones.

Tabla 1. Operaciones por elemento, N = 1 000 (µs)

Estructura	Insert	Member (presente)	Member (ausente)	Delete
Lista por punteros	7,3	18,4	17,9	12,1
Lista por arreglos	12,0	12,0	12,0	14,9
Tabla hash (m = 10)	0,92	0,90	1,39	1,34
Tabla hash (m = N), referencia	0,51	0,50	0,46	0,89
ABB por punteros	0,77	0,58	0,70	0,73
ABB por vector heap	inviable	inviable	inviable	inviable
Trie por punteros	2,78	1,81	1,13	9,18
Trie por arreglos	2,54	1,63	0,86	5,30

Tabla 2. Operaciones por elemento, N = 10 000 (µs)

Estructura	Insert	Member (presente)	Member (ausente)	Delete
Lista por punteros	80,1	214,4	251,7	152,3
Lista por arreglos	134,4	105,7	114,8	152,3
Tabla hash (m = 10)	4,68	4,79	8,99	8,96
Tabla hash (m = N), referencia	0,56	0,61	0,60	0,70
ABB por punteros	0,77	0,86	1,04	1,59
ABB por vector heap	inviable	inviable	inviable	inviable
Trie por punteros	3,76	2,86	1,68	9,97
Trie por arreglos	3,21	2,69	1,43	7,30

Tabla 3. Operaciones sobre el conjunto completo, N = 10 000 (milisegundos)

Estructura	Union	Intersection	Difference	Print
Lista por punteros	3 842	2 624	3 203	3,6
Lista por arreglos	6 864	2 124	1 764	1,7
Tabla hash (m = 10)	134,9	81,1	80,3	1,9
Tabla hash (m = N), referencia	12,1	9,0	8,8	6,3
ABB por punteros	15,9	12,7	12,8	3,4
ABB por vector heap	inviable	inviable	inviable	–
Trie por punteros	146,3	92,2	93,9	41,9
Trie por arreglos	209,3	80,9	136,6	44,3

Tabla 4. Operaciones sobre el conjunto completo, N = 1 000 (milisegundos)

Estructura	Union	Intersection	Difference	Print
Lista por punteros	33,7	23,0	23,3	0,20
Lista por arreglos	60,6	19,0	18,5	0,20
Tabla hash (m = 10)	2,10	1,51	1,43	0,20
Tabla hash (m = N), referencia	1,77	1,29	1,33	0,62
ABB por punteros	1,62	1,34	1,33	0,39
ABB por vector heap	inviable	inviable	inviable	–
Trie por punteros	15,9	7,9	8,7	5,0
Trie por arreglos	20,9	7,6	13,7	4,3

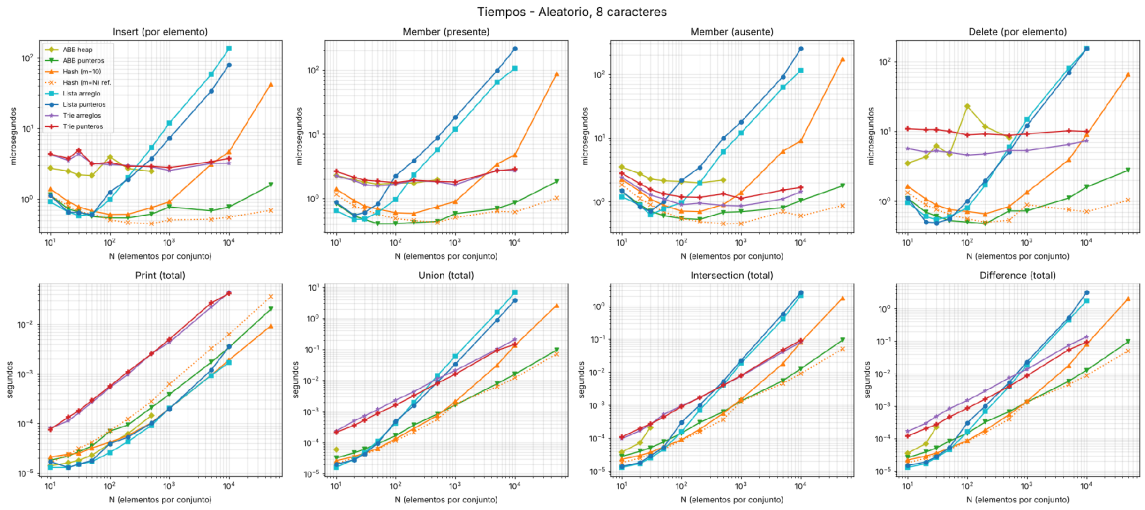


Figura 1. Tiempo de cada operación según N con el conjunto aleatorio de 8 caracteres (ejes logarítmicos). El ABB por vector heap termina antes porque deja de ser viable.

Lectura por estructura.

- **Listas.** Al multiplicar N por 10 (de 1 000 a 10 000), Insert y Member se multiplican por 10 u 11, lo que confirma un costo lineal por operación. Union se multiplica por cerca de 114, lo que confirma un costo cuadrático: cada elemento de A y de B se inserta recorriendo el resultado desde el inicio. En Member la lista por punteros es más lenta que la de arreglos (214 µs frente a 106 µs con N = 10 000) porque seguir referencias entre nodos es más costoso que recorrer posiciones contiguas. En Insert ocurre lo contrario (80 µs frente a 134 µs), porque el arreglo además desplaza elementos.
- **Tabla hash con m = 10.** Las cadenas de cada posición miden aproximadamente N/10, así que el costo por operación crece de forma lineal con N (Insert: 0,92 µs con N = 1 000, 4,7 µs con 10 000

y 42 µs con 50 000). Con m = N el costo por operación se mantiene alrededor de 0,5 µs y Union crece linealmente con N.

- **ABB por punteros.** Con datos aleatorios Insert pasa de 0,77 µs (N = 1 000) a 1,58 µs (N = 50 000), crecimiento logarítmico. Union, Intersection y Difference crecen linealmente (de 1,6 ms a 15,9 ms al multiplicar N por 10) porque mezclan los recorridos inorden de A y B y construyen un árbol balanceado sin lista auxiliar.
- **Tries.** Insert y Member casi no cambian de N = 1 000 a N = 10 000 (por ejemplo, Insert en el trie por arreglos pasa de 2,5 µs a 3,2 µs), porque dependen de la longitud de la cadena. Delete es más caro (5 a 10 µs) porque guarda el camino y poda los nodos vacíos. Union crece linealmente con el número de nodos. Print es la operación más cara de los tries (unos 42 ms con N = 10 000, contra 2 a 6 ms en las demás estructuras), porque recorre 36 posiciones por nodo e imprime un elemento por línea.

Tabla 5. Comportamiento observado al multiplicar N por 10 (aleatorio 8, de N = 1 000 a N = 10 000)

Estructura	Insert, Member y Delete	Union, Intersection y Difference	Orden observado
Listas	× 10 a × 12	× 100 a × 115	O(N) por operación y O(N^2) en conjunto
Hash (m = 10)	× 5 a × 7	× 54 a × 64	O(N / m) por operación
Hash (m = N)	× 1	× 7	O(1) por operación y O(N) en conjunto
ABB por punteros	× 1 a × 2	× 10	O(log N) por operación y O(N) en conjunto
Tries	× 1,2 a × 1,7	× 8 a × 13	O(L) por operación y O(nodos) en conjunto

4. Efecto del tipo de conjunto

Las tablas 6 y 7 comparan los cuatro tipos con N = 10 000. Los gráficos completos están en las figuras 2 a 4.

Tabla 6. Insert por elemento, N = 10 000 (µs)

Estructura	Aleatorio 8	Aleatorio 20	Prefijo común	Ordenado
Lista por punteros	80,1	74,3	76,5	136,0
Lista por arreglos	134,4	138,2	132,4	289,6
Tabla hash (m = 10)	4,68	5,39	6,18	4,41
ABB por punteros	0,77	0,96	0,92	**222,3**
Trie por punteros	3,76	9,88	5,36	3,34
Trie por arreglos	3,21	9,15	3,94	2,68

Tabla 7. Union, N = 10 000 (milisegundos)

Estructura	Aleatorio 8	Aleatorio 20	Prefijo común	Ordenado
Lista por punteros	3 842	4 004	3 825	4 105
Lista por arreglos	6 864	7 124	6 726	6 892
Tabla hash (m = 10)	134,9	140,0	140,5	115,9
ABB por punteros	15,9	18,8	17,8	15,2
Trie por punteros	146,3	490,7	175,8	144,2
Trie por arreglos	209,3	632,1	108,3	182,3

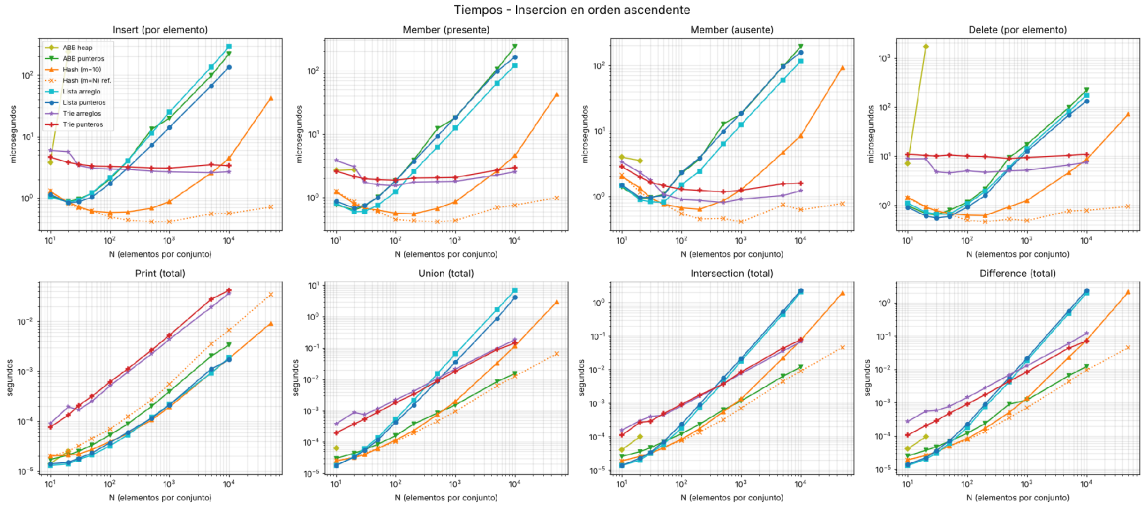


Figura 2. Tiempos con inserción en orden ascendente: el ABB por punteros pierde su ventaja en Insert, Member y Delete.

- **Orden de inserción.** Es lo que más afecta al ABB por punteros. Con inserción ordenada el árbol se vuelve una cadena y Insert, Member y Delete pasan de unos 0,8 µs a más de 200 µs con N = 10 000 (el costo por operación crece de 20 µs con N = 1 000 a 98 µs con N = 5 000 y 222 µs con N = 10 000). Con N = 50 000 la estimación superó el presupuesto de tiempo y no se midió. Las operaciones de conjunto no se ven afectadas, porque construyen el resultado balanceado. La lista por arreglos también duplica su costo de Insert, porque cada elemento nuevo va al final y obliga a recorrer toda la lista.
- **Longitud de las cadenas.** Las cadenas de 20 caracteres multiplican por unas 2,5 a 3 veces el costo de Insert y Union en los tries (el costo depende de L, con $L \leq 20$), mientras que las demás estructuras casi no cambian (solo la comparación de cadenas más largas).
- **Prefijo común.** Los tries son los que más se benefician: comparten los nodos del prefijo y ocupan cerca de la mitad de memoria que con cadenas aleatorias de 8 caracteres (1,2 KB frente a 2,4 KB por elemento). En los demás casos los tiempos son similares.

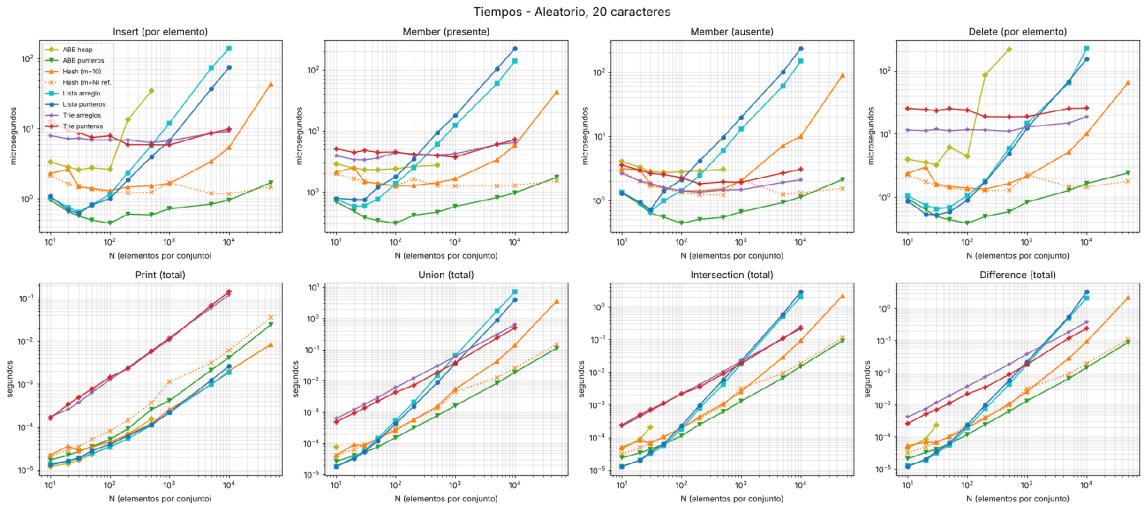


Figura 3. Tiempos con cadenas aleatorias de 20 caracteres: los tries son los que más sufren el aumento de longitud.

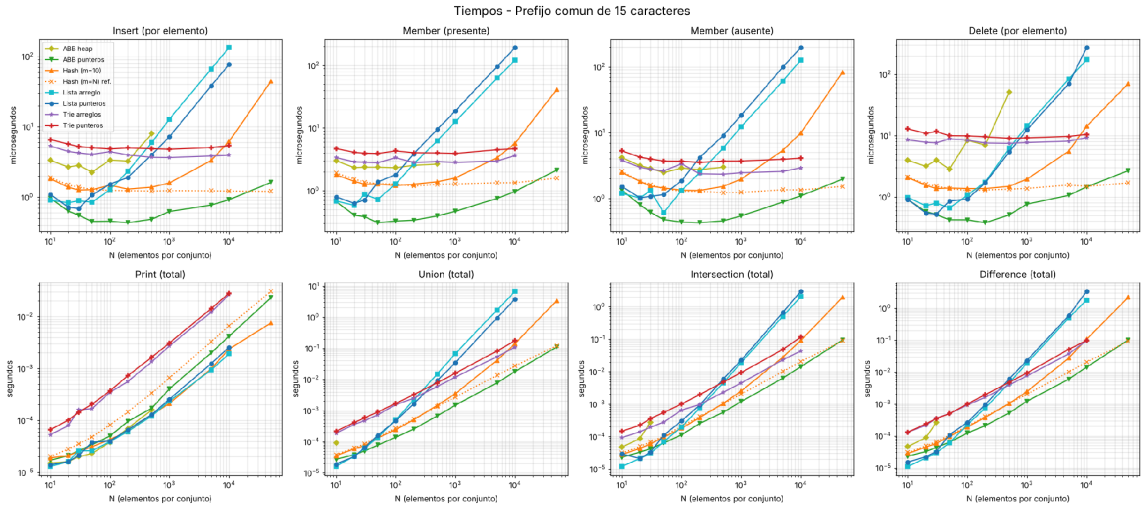


Figura 4. Tiempos con prefijo común de 15 caracteres: los costos son parecidos a los del caso aleatorio y los tries ahorran memoria.

5. Resultados de espacio

Tabla 8. Bytes por elemento, N = 10 000 (incluye la cadena del elemento)

Estructura	Aleatorio 8	Aleatorio 20	Prefijo común	Ordenado
Lista por arreglos	58	70	70	58
Tabla hash (m = 10)	58	70	70	58
Lista por punteros	129	141	141	129
Tabla hash (m = N), referencia	134	146	146	134
ABB por punteros	137	149	149	137
Trie por punteros	2 364	7 065	1 189	2 364
Trie por arreglos	2 557	7 642	1 286	2 557

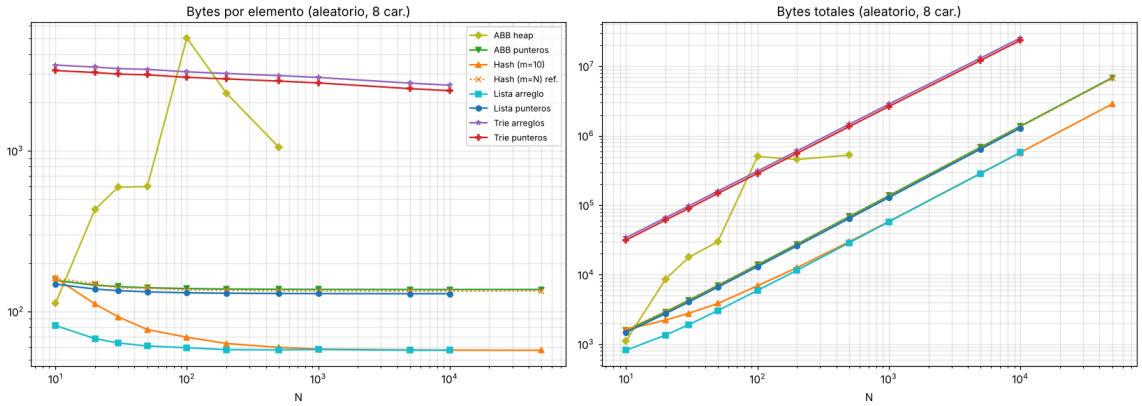


Figura 5. Uso de espacio con el conjunto aleatorio de 8 caracteres: bytes por elemento (izquierda) y bytes totales (derecha).

- **Arreglos y tabla hash de 10 posiciones** son los más compactos: unos 8,7 bytes por elemento más la cadena (en Python 3.12, una cadena de 8 caracteres ocupa 49 bytes). La tabla hash conserva ese consumo porque sus listas internas son arreglos de Python.
- **Estructuras con nodos** (lista por punteros, ABB por punteros) gastan alrededor de 80 bytes por nodo además de la cadena: el objeto, su diccionario interno y los punteros.
- **Tries.** Cada nodo reserva un arreglo de 36 referencias (unos 290 bytes) y la mayor parte de las cadenas aleatorias no comparten nodos más allá de los primeros niveles, por lo que cada elemento necesita varios nodos propios. Con N = 10 000 y cadenas de 20 caracteres, un solo conjunto ocupa unos 71 MB (trie por punteros) o 76 MB (trie por arreglos); la lista por arreglos ocupa 0,7 MB. El trie por punteros usa entre 5 % y 8 % menos memoria que el de arreglos en todos los casos, porque sus nodos usan `__slots__`.
- **ABB por vector heap.** Su espacio depende de la forma del árbol, no de N. La tabla 9 muestra el vector requerido con datos aleatorios de 8 caracteres.

Tabla 9. ABB por vector heap (aleatorio 8): tamaño del vector y operaciones de conjunto

N	Posiciones del vector	Bytes por elemento	Union	Intersection	Difference
10	38	112	0,06 ms	0,04 ms	0,04 ms
30	2 004	593	inviable	0,21 ms	0,23 ms
100	62 234	5 031	inviable	inviable	inviable
500	62 769	1 054	inviable	inviable	inviable
1 000	más de 4 194 304	inviable	inviable	inviable	inviable

Con inserción ordenada el vector requiere 2^N posiciones: con N = 20 ya son 1 048 575 y con N = 30 ocupa 2^30.

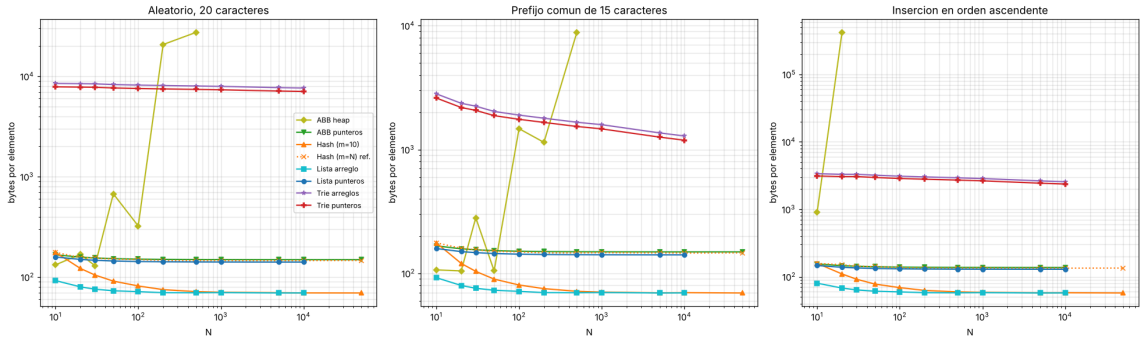


Figura 6. Bytes por elemento en los otros tipos de conjunto: cadenas de 20 caracteres, prefijo común y orden ascendente.

6. Rangos de N medidos y recomendados

Tabla 10. Rango de N por estructura

Estructura	N máximo medido	Rango recomendado según las mediciones	Factor limitante
Lista por punteros	10 000	Hasta unos 5 000 (Union ≈ 0,9 s)	Union, Intersection y Difference cuadráticas; con N = 10 000 ya toma 3,8 s
Lista por arreglos	10 000	Hasta unos 5 000 (Union ≈ 1,5 s)	Igual que la anterior, con Insert más lento por el desplazamiento
Tabla hash (m = 10)	50 000	Hasta unos 10 000 (Union ≈ 135 ms)	Tamaño fijo: el costo crece con N/10; con N = 50 000 Union ≈ 2,6 s

Estructura	N máximo medido	Rango recomendado según las mediciones	Factor limitante
Tabla hash (m = N), referencia	50 000	Más de 50 000	Ninguno observado
ABB por punteros	50 000 (aleatorio) y 10 000 (ordenado)	Más de 50 000 con orden aleatorio; hasta unos 5 000 con orden ascendente	Degeneración con datos ordenados
ABB por vector heap	500 (operaciones básicas)	Hasta unos 100 con datos aleatorios; Union hasta N ≈ 10 e Intersection y Difference hasta N ≈ 30	Crecimiento exponencial del vector con la altura del árbol
Trie por punteros	10 000	Hasta 10 000 medido; el tiempo crece linealmente y la memoria (1,2 a 7,1 KB por elemento) es el límite	Memoria: unos 70 MB por conjunto con N = 10 000 y cadenas de 20 caracteres
Trie por arreglos	10 000	Igual que el anterior	Igual que el anterior

Los valores de «recomendado» son un criterio práctico basado en las mediciones (Union por debajo de aproximadamente 2 s y Delete por debajo de 1 ms). Dependen del equipo y se pueden recalcular con `analisis.py`.

7. Limitaciones

- Python agrega un costo fijo por objeto y por llamada, así que las diferencias entre estructuras con costo similar (por ejemplo, ABB por punteros frente a tabla hash con m = N) son menores que su significado teórico. Las tendencias de crecimiento, en cambio, sí coinciden con las complejidades esperadas.
- Los tiempos se midieron en un solo equipo por tipo de conjunto y tamaño. Los valores absolutos no son transferibles.
- Member y Delete se miden con muestras (hasta 1 000 y 300 elementos) para no tardar horas en las listas.
- Solo se probó un solapamiento del 50 % entre A y B. Con solapamientos distintos cambian los tamaños de los resultados de Union, Intersection y Difference.
- `tracemalloc` mide la memoria asignada por Python, no la memoria residente del proceso.
- Por tiempo se omitieron N = 50 000 en las listas y el ABB por punteros con datos ordenados, y por memoria los tries quedaron limitados a N = 10 000. Para estos casos solo se dispone de la estimación.

8. Conclusiones

- Con datos de orden aleatorio, el **ABB por punteros** y la **tabla hash con redistribución** tienen el mejor tiempo en todas las operaciones. El ABB además produce resultados ordenados y soporta mejor la memoria (137 bytes por elemento).
- La **tabla hash de tamaño fijo** y las **listas** sirven para conjuntos pequeños. A partir de unos 5 000 elementos las listas se vuelven poco prácticas para las operaciones de conjunto.
- Los **tries** tienen el comportamiento más predecible (dependen de la longitud de la cadena, no de N ni del orden de inserción) y son mejores cuando los elementos comparten prefijos, pero cuestan mucha memoria y su Print es el más lento.
- El **ABB por vector heap** cumple con el modelo conjunto solo para conjuntos muy pequeños. Con los cambios de la sección 8 se podría ampliar su rango.
- El orden de inserción importa más que el tamaño en las estructuras sin balanceo. Los resultados confirman las complejidades teóricas estudiadas: lineal y cuadrática en las listas, logarítmica en el ABB aleatorio, constante esperada en la tabla hash con tamaño adecuado y proporcional a la longitud en los tries.